TunaTrace Live Deployment Guide

From a free test wallet to a public link your lecturer can use

22 October 2026 · Companion to the report *Consensus, Scaling and Trust: Designing TunaTrace* (Section 7.5)

This guide takes TunaTrace from your laptop onto a public test blockchain (**Ethereum Sepolia**) and gives you one public link your lecturer can open. On that page he or she can check packs of tuna as a shopper, view every catch and lot with its history straight from the blockchain, and create real entries (declare a catch, pack a lot, ship, record temperatures, run a DNA test, sell, suspend a vessel, recall) without installing anything.

You never need real money. Everything uses free test coins that have no value.

How it fits together

Part	What it is	Where it lives
Your MetaMask wallet	Your blockchain account. It pays the (free, test) fees to put the contract on Sepolia and becomes the contract's administrator and fisheries authority.	Browser extension on your laptop
The TunaTrace contract	The same contract you tested (25 tests), now running on Sepolia with a permanent public address.	Sepolia test network
Six demo wallets	Accounts for the regulator, processor, distributor, retailer, IoT gateway and laboratory. The deploy script creates them and gives each a little test ETH.	Sepolia; keys in `web/deployment.js`
One vessel key	Stands in for the fishing vessel's monitoring unit. It only *signs* catch declarations, so it never needs ETH.	`web/deployment.js`
`web/live.html`	The live page. It reads the contract for everyone and uses the demo wallets to send transactions, so visitors need no wallet.	GitHub Pages (your public link)
`web/index.html`	The offline console, which runs the same rules in the browser and still works without a blockchain.	GitHub Pages

The demo keys are deliberately public, so anyone can act as each organisation. That is safe because they only ever hold small amounts of test ETH. **Your own MetaMask key is never put in any file.**

Time needed: about 1.5 to 2 hours, plus waiting for the faucet. Do Steps 1 to 4 a day before the deadline in case the faucet is slow.

Step 1: Install MetaMask

1. Open Google Chrome or Microsoft Edge.
2. Go to **metamask.io** and click **Download**. Only use the official site; fake MetaMask extensions exist.
3. Click **Add to Chrome** (or **Get** in Edge), then **Add extension**.

4. Click the puzzle-piece icon at the top right of the browser and pin MetaMask so its fox icon is always visible.

Step 2: Create a new wallet for this project

1. Click the fox icon and choose **Create a new wallet**.
2. Agree to the terms and choose a password. This password only unlocks MetaMask on this laptop.
3. MetaMask shows your **Secret Recovery Phrase** (12 words). Write the words on paper, in order, and keep the paper safe. Never type them into a website, a document or a chat, and never share them with anyone.
4. Confirm the phrase as MetaMask asks, then finish.

> **Rules for this wallet.** Use it only for this project and test networks. Never send real cryptocurrency to it. If anyone asks for your recovery phrase, it is a scam.

Step 3: Switch MetaMask to the Sepolia test network

1. Click the fox icon, then the network name at the top left (it usually says *Ethereum*).
2. Turn on **Show test networks** (or open *Settings* → *Advanced* and turn it on).
3. Select **Sepolia**. Your balance shows 0 SepoliaETH.
4. Click your account name or the copy icon to copy your **address**. It starts with `0x` and is 42 characters long. Your address is public and safe to share, like an account number.

Step 4: Get free test ETH from a faucet

A faucet is a website that gives out free test coins. You need roughly **0.02 to 0.06 SepoliaETH**; the deploy script tells you the exact amount in Step 7. Try these in order until one works:

Faucet	Address	Notes
Google Cloud Web3 faucet	cloud.google.com/application/web3/faucet/ethereum/sepolia	Sign in with a Google account, paste your address, click *Receive*.
QuickNode faucet	faucet.quicknode.com/ethereum/sepolia	Paste your address; the base amount needs no account.
Sepolia PoW faucet	sepolia-faucet.pk910.de	Paste your address and click *Start Mining*. Leave the tab open for a while, then *Stop Mining* and *Claim*.

1. Open the faucet, paste your address from Step 3 and request the test ETH.
2. Wait 1 to 5 minutes, then check MetaMask. The SepoliaETH balance should go up.

Faucet rules change often. If one asks you to pay or to hold real ETH, skip it and use another.

Step 5: Get the project and check it works

1. Download **tunatrace-source.zip**. Right-click it → **Extract All** → choose **Documents** → **Extract**.
2. Open `Documents\tunatrace`, right-click an empty area → **Open Git Bash here** (install Node.js 18+ and Git for Windows first if you have not).
3. Install the libraries and run the tests:

```
npm install --legacy-peer-deps
npm test
```

You should see **25 passing**. Optionally run `npm run simulate` to regenerate `reports/simulation.json`.

Step 6: Give Hardhat your wallet's key (safely)

Hardhat needs your MetaMask account's private key to sign the deployment. Store it with Hardhat's built-in secret store, which keeps it in a settings folder on your laptop *outside* the project folder, so it can never be uploaded to GitHub by mistake.

1. **Copy the private key from MetaMask:** fox icon → three dots next to your account (or the account name) → **Account details** → **Show private key** (in some versions under *Private keys*) → enter your MetaMask password → copy.
2. **Store it in Hardhat.** In Git Bash type the line below, replacing `PASTE_YOUR_KEY_HERE` with your key (paste with Shift+Insert), then press Enter:

```
npx hardhat vars set SEPOLIA_PRIVATE_KEY PASTE_YOUR_KEY_HERE
clear
```

Check with `npx hardhat vars list`: you should see `SEPOLIA_PRIVATE_KEY` (the key itself is not shown).

> Never paste the private key into the report, a chat, GitHub or any file inside the project. The private key controls the wallet; the recovery phrase controls all its accounts.

Step 7: Deploy TunaTrace to Sepolia

```
npx hardhat run scripts/deploy-live.js --network sepolia
```

What happens. The script first prints a summary and checks your balance:

```
Network:        sepolia (chain 11155111)
Your account:   0x...your address...
Balance:        0.05000 ETH
Gas price now:  1.2 gwei
Estimated need: 0.02240 ETH
```

Then it carries out 36 numbered steps, waiting about 12 seconds for each to be confirmed. The whole run takes **7 to 9 minutes**; leave the window alone until it finishes.

```
[01] Contract deployed at 0x1234...abcd (block 7654321)
[02] Funded regulator wallet 0x5c61... with 0.00108 ETH
...
[16] Processor declared catch: 600 kg YFT at -19.2514, 58.1032 (signed by the vessel)
[17] Processor packed lot YFT-ULT-2610-A (100 x 300 g frozen saku)
...
[30] YFT-ULT-2610-B window 4: -30 to -26 C
[35] Retailer sold pack TT0042
[36] YFT-ULT-2610-B status on-chain: Quarantined

Done. Contract: 0x1234...abcd
View it:        https://sepolia.etherscan.io/address/0x1234...abcd
Page config:    web/deployment.js (open web/live.html)
```

Steps	What the script does
01	Puts the contract on Sepolia; your MetaMask account becomes the administrator and fisheries authority
02–07	Creates the six demo wallets and sends each some test ETH (the vessel key needs none)
08–13	Gives each demo wallet its role (regulator, processor, distributor, retailer, oracle, inspector)
14–15	Licenses the vessel *FV Albacora Star* for a zone around Mauritius and declares a coastal marine park as a no-take zone
16	The processor submits a 600 kg yellowfin catch declaration signed by the vessel's key
17–22	Packs two lots of 100 frozen saku blocks (−60 °C class), fits temperature loggers and ships both to the distributor
23–30	Sends 4 signed temperature windows per lot: lot A has a short dock spike (within budget); lot B's reefer compressor fails
31–35	The lab confirms lot A is yellowfin (DNA); lot A reaches the supermarket and pack TT0042 is sold
36	Confirms lot B was quarantined automatically by the contract

When it finishes it has created `web\deployment.js` (contract address, interface and demo keys, which the live page needs) and a summary in `reports\deployment-sepolia.json`. **Copy the contract address into a note**; you will put it in your report.

> The demo lots are **frozen** product with an 18-month use-by date on purpose, so the shopper samples stay valid while the work is being marked. Chilled lots you create yourself expire after about 10 days, exactly as the contract's freshness rule intends.

If something goes wrong

Message	What to do

Not enough test ETH. Get at least … more	Get more from another faucet (Step 4), then run the same command again. Nothing has been deployed yet.
No deployer key	Redo Step 6; check the spelling `SEPOLIA_PRIVATE_KEY`.
A network or timeout error part-way	Run the command again. It deploys a fresh contract and writes a new `deployment.js`; the half-finished one is simply ignored.

Step 8: See your contract on Etherscan

1. Open the *View it* link the script printed, for example `https://sepolia.etherscan.io/address/0x1234...abcd`.
2. Under **Transactions** you see the contract creation and every role grant, the vessel licence, the catch, both lots and every temperature report. Each one has a transaction hash, a block number and a time.
3. Click any transaction to see who sent it and that it succeeded.

This is your proof that the system runs on a real public blockchain. **Take a screenshot for your report.**

Step 9: Try the live page on your laptop

1. In File Explorer open `tunatrace\web` and double-click **live.html**.
2. It connects to Sepolia (the header shows a block number and your contract address) and lists the two lots and the catch.
3. Open the **Shopper check** tab: the four sample buttons should give *Genuine tuna*, *Already sold*, *Not recognised* and *Do not eat*.
4. In the **Stakeholder portal** select lot `YFT-ULT-2610-B` to see its temperature chart, abuse budget and history, read from the blockchain.

Each action on this page is a real transaction, so it takes about 12 seconds per step and appears on Etherscan. A box in the bottom corner shows progress and links to each transaction. Actions the contract would refuse are checked first for free and explained in plain words, so they cost nothing.

Try this	As	What happens
Declare a catch with the *Legal catch* preset	Processor	A new catch appears under *Declared catches* after one transaction
Press *Inside marine park* or *Outside licence zone*, then submit	Processor	Refused before sending: illegal position (no gas spent)
Pack a lot (frozen), then try 999 kg of raw fish	Processor	First works; second refused as possible laundering (mass balance)
Ship to distributor, then send temperature reports with a peak of −30 °C for 45 minutes	Processor, then Distributor	Four transactions (logger + 3 windows); the lot becomes *Quarantined*
Release after histamine test	Lab / inspector	Lot returns to the processor; the abuse stays on record and the use-by date stays shortened

Result: substituted species	Lab / inspector	DNA mismatch quarantines the lot
Accept, ship to retailer, accept	Distributor, then Retailer	Custody moves along the chain
Sell TT0007, then TT9999	Retailer	First works; second refused as a possible counterfeit
Suspend (IUU listing), then try to declare a catch; then Reinstate	Fisheries authority, Processor	Catches are refused while the vessel is suspended
Recall a lot	Fisheries authority	Every pack of that lot now shows *Recalled* in the shopper check

Step 10: Create your GitHub account and repository

1. Go to **github.com/signup** and create a free account. Your username becomes part of your public link.
2. Go to **github.com/new**: repository name `tunatrace`; select **Public**; tick **Add a README file**; click **Create repository**.

Step 11: Upload the project

1. In your new repository click **Add file → Upload files**.
2. In File Explorer open `Documents\tunatrace` and select everything **except** the `node_modules`, `artifacts` and `cache` folders. Check that `web\deployment.js` is included, together with `style.css`, `app.js`, `chain-live.js` and `chain-sim.js`; the live page needs them all.
3. Drag the selection onto the GitHub page, wait until every file is listed, then click **Commit changes**.

Your private key is not in any of these files: it is stored in Hardhat's settings folder (Step 6). The only keys in the upload are the demo keys in `deployment.js`, which are meant to be public.

Step 12: Turn on GitHub Pages (your public link)

1. In the repository click **Settings**, then **Pages**.
2. Under *Build and deployment → Branch* choose **main** and **/ (root)**, then **Save**.
3. Wait 1 to 2 minutes and refresh. GitHub shows *Your site is live at https://yourusername.github.io/tunatrace/*.

Page	Link
Live blockchain system (give this to your lecturer)	https://yourusername.github.io/tunatrace/web/live.html
Offline console	https://yourusername.github.io/tunatrace/web/
Source code	https://github.com/yourusername/tunatrace
Contract on Etherscan	https://sepolia.etherscan.io/address/<your contract address>

> **Real QR codes.** Once the page is hosted, select any lot in the portal: the *Pack label* QR code now contains your public link. Scan it with a phone camera and the shopper check opens and verifies that pack. Print one label and scan it for a striking screenshot for your report.

Step 13: Test it the way your lecturer will

1. Open the live link in a private window (Ctrl+Shift+N), so nothing from your own browser helps it.
2. Check that the header shows a block number and that the lots and the catch load.
3. Do one quick action, for example declaring a legal catch as the processor, and check that it appears.
4. Try it on your phone too, including scanning a label QR code.

If the page says it cannot reach the blockchain, refresh once; it automatically tries three public connection points.

Step 14: Add it to your report

Complete the highlighted paragraph in **Section 7.5** of the report:

> *TunaTrace was deployed to the Ethereum Sepolia test network at contract address 0x... (transaction history: https://sepolia.etherscan.io/address/0x...). A live interface is available at https://yourusername.github.io/tunatrace/web/live.html. It reads catches, lots, temperature history and pack verdicts directly from the contract, and lets visitors act as each organisation through demo accounts funded with test ETH, so no wallet is needed. Sepolia is used as a stand-in for the permissioned QBFT network proposed in Section 4.3; the contract is identical, only the network differs. The source code is at https://github.com/yourusername/tunatrace.*

Also add screenshots of: the deploy script's final lines; the contract on Etherscan; the live page showing the quarantined lot; and a shopper check (ideally opened by scanning a QR code). Put the live link near the start of **Section 6.3** as well, so your lecturer can click it while reading.

> **This sentence matters for marking:** Sepolia is a public Proof-of-Stake network, while your design recommends a permissioned QBFT consortium. Saying that Sepolia is used only to demonstrate the contract publicly shows that you understand the difference.

Step 15: Keep it working until it's marked

- If the page says a demo wallet has run out of test ETH, get more test ETH into your MetaMask (Step 4), then run:

  ```
  npx hardhat run scripts/topup-live.js --network sepolia
  ```

  This refills each demo wallet; you do not need to upload anything again.
- If you redeploy (Step 7 again), a new `web/deployment.js` is created. Upload that one file to the `web` folder on GitHub (*Add file → Upload files → Commit changes*) and update the contract address in your report.

- Sepolia is a test network. It is meant to run for years, but test networks are sometimes retired with notice. The offline console (`web/index.html`) and your screenshots remain as backup evidence.
- After the module is marked, remove the key from your laptop with `npx hardhat vars delete SEPOLIA_PRIVATE_KEY`.

Optional: rehearse everything locally first

You can run the whole live page against a private local blockchain before touching Sepolia, with no faucet and no waiting:

```
# terminal 1
npx hardhat node
# terminal 2
npx hardhat run scripts/deploy-live.js --network localhost
npx http-server web -p 8080     # then open http://127.0.0.1:8080/live.html
```

Running Step 7 afterwards overwrites `web/deployment.js` with the Sepolia version, which is the one to upload.

Quick reference: all commands in order

```
# Step 5: in Documents/tunatrace
npm install --legacy-peer-deps
npm test

# Step 6: store your MetaMask TEST key (outside the project)
npx hardhat vars set SEPOLIA_PRIVATE_KEY PASTE_YOUR_KEY_HERE
clear

# Step 7: deploy to Sepolia
npx hardhat run scripts/deploy-live.js --network sepolia

# Step 15: refill demo wallets if needed
npx hardhat run scripts/topup-live.js --network sepolia
```